Python Programming Fundamentals

Methods and Validation

Programming Fundamentals Team

SETU

2026-06-12

Outline

Outline

1. Adding More Methods

Methods can call other methods using `self.method_name()`.

```
class Rectangle:
    def __init__(self, width, height):
        self.width  = width
        self.height = height

    def area(self):
        return self.width * self.height

    def perimeter(self):
        return 2 * (self.width + self.height)

    def is_square(self):
        return self.width == self.height
```

1. Adding More Methods

```
    def scale(self, factor):
        """Return a new Rectangle scaled by factor."""
        return Rectangle(self.width * factor, self.height * factor)

    def __str__(self):
        return f"Rectangle({self.width} x {self.height})"

r = Rectangle(4, 6)
print(r)                    # Rectangle(4 x 6)
print(f"Area: {r.area()}")
print(f"Perimeter: {r.perimeter()}")
print(f"Is square: {r.is_square()}")
big = r.scale(3)
print(big)                  # Rectangle(12 x 18)
```

Outline

2. Default Parameters in Methods

Methods can have default parameter values, just like regular functions.

```
class BankAccount:
    def __init__(self, name, balance=0, currency="EUR"):
        self.name     = name
        self.balance  = balance
        self.currency = currency

    def deposit(self, amount, description="Deposit"):
        if amount <= 0:
            raise ValueError("Amount must be positive")
        self.balance += amount
        print(f"{description}: +{amount:.2f} {self.currency}")

    def withdraw(self, amount, description="Withdrawal"):
        if amount <= 0:
```

2. Default Parameters in Methods

```
            raise ValueError("Amount must be positive")
        if amount > self.balance:
            raise ValueError("Insufficient funds")
        self.balance -= amount
        print(f"{description}: -{amount:.2f} {self.currency}")

acc = BankAccount("Alice", 1000)
acc.deposit(500)                          # uses default description
acc.deposit(200, "Salary")                # custom description
acc.withdraw(100, description="Rent")
```

Outline

3. Validation in __init__

Validate data **as soon as it is set** — in `__init__`.

```
class Student:
    def __init__(self, name, student_id, gpa):
        # Validate name
        if not name or not name.strip():
            raise ValueError("Name cannot be empty")
        self.name = name.strip()

        # Validate student_id
        if not student_id.startswith("S"):
            raise ValueError("Student ID must start with 'S'")
        self.student_id = student_id

        # Validate GPA
        if not 0.0 <= gpa <= 4.0:
```

3. Validation in __init__

```
            raise ValueError(f"GPA must be between 0.0 and 4.0, got {gpa}")
        self.gpa = gpa

    def __str__(self):
        return f"Student({self.student_id}: {self.name}, GPA={self.gpa:.2f})"
```

Outline

4. Raising ValueError

`raise ValueError("message")` signals that invalid data was provided.

```
# Good validation pattern
def set_age(self, age):
    if not isinstance(age, int):
        raise ValueError(f"Age must be an integer, got {type(age).__name__}")
    if age < 0:
        raise ValueError(f"Age cannot be negative, got {age}")
    if age > 150:
        raise ValueError(f"Age {age} is unrealistically large")
    self.age = age
```

4. Raising ValueError

4. Raising ValueError

`raise ValueError("message")` signals that invalid data was provided.

```
# Good validation pattern
def set_age(self, age):
    if not isinstance(age, int):
        raise ValueError(f"Age must be an integer, got {type(age).__name__}")
    if age < 0:
        raise ValueError(f"Age cannot be negative, got {age}")
    if age > 150:
        raise ValueError(f"Age {age} is unrealistically large")
    self.age = age
```

The caller handles the exception:

```
try:
    s = Student("Alice", "S001", 5.0)    # GPA too high
except ValueError as e:
```

4. Raising ValueError

```
    print(f"Cannot create student: {e}")

try:
    s = Student("", "S002", 3.5)          # empty name
except ValueError as e:
    print(f"Cannot create student: {e}")

# This one is valid
s = Student("Bob", "S003", 3.8)
print(s)
```

Outline

5. Testing Your Classes

Good practice: test each method thoroughly.

```
def test_bank_account():
    """Test BankAccount class."""
    print("Testing BankAccount...")

    # Test 1: Create account
    acc = BankAccount("ACC001", "Alice", 1000)
    assert acc.balance == 1000, "Initial balance wrong"
    print("  PASS: Initial balance")

    # Test 2: Deposit
    acc.deposit(500)
    assert acc.balance == 1500, "Deposit failed"
    print("  PASS: Deposit")
```

5. Testing Your Classes

```
    # Test 3: Withdraw
    acc.withdraw(200)
    assert acc.balance == 1300, "Withdraw failed"
    print("  PASS: Withdraw")

    # Test 4: Overdraft rejected
    try:
        acc.withdraw(99999)
        print("  FAIL: Should have raised ValueError")
    except ValueError:
        print("  PASS: Overdraft rejected")

    print("All tests passed!")

test_bank_account()
```

Outline

6. Class-Level Variables (Class Attributes)

Instance variables (on `self`) belong to each object individually.

Class variables belong to the class and are shared by all instances.

```
class BankAccount:
    interest_rate = 0.02    # class variable — shared by all accounts
    account_count = 0       # track how many accounts exist

    def __init__(self, name, balance=0):
        self.name    = name
        self.balance = balance
        BankAccount.account_count += 1  # increment class variable

    def apply_interest(self):
        interest = self.balance * BankAccount.interest_rate
        self.balance += interest
```

6. Class-Level Variables (Class Attributes)

```
        return interest

acc1 = BankAccount("Alice", 1000)
acc2 = BankAccount("Bob", 2000)
acc3 = BankAccount("Carol", 1500)

print(f"Total accounts: {BankAccount.account_count}")  # 3
print(f"Alice interest: €{acc1.apply_interest():.2f}")

# Change rate for ALL accounts at once
BankAccount.interest_rate = 0.03
```

Thanks for Watching - Any questions?